



Python程序设计

一、基础知识

学习内容：

1. 模块和包

- Import使用：math模块，random模块

1.4.9 模块(modules)导入与使用

- 相互之间有相应联系的一些函数以及变量组织在一起，放到同一个Python源文件(.py)，这个.py文件就是一个**模块 (module)**
 - 提高了代码的可维护性和可重用性
 - 不同模块属于不同的名字空间，可以避免函数名和变量名冲突。
- Python启动时，Python默认安装仅包含基本或核心模块，启动时也仅加载了基本模块，仅加载了很少的一部分模块，在需要时由程序员显式地加载（可能需要先安装）其他模块。如在需要时再**显式地导入和加载**标准库(如math, random等)

这样做的目的是减小运行的压力，仅加载真正需要的模块和功能，且具有很强的可扩展性。
- 可以使用**sys.modules.items()**显示所有预加载模块的相关信息。
- **sys.builtin_module_names**给出了Python解释器内置的模块
- 需要使用**import sys**

模块的使用

- **import 模块名 [as 别名]** :

- 访问模块中的对象，采用**模块名.对象**或者**别名.对象**方式

```
>>>import math
```

```
>>>math.sin(0.5)           #求0.5的正弦
```

```
>>>import random as rnd
```

```
>>>x=rnd.random( )  #获得[0,1) 内的随机小数
```

```
>>> import random as rnd
```

```
>>> rnd.random()
```

```
0.27259739220371226
```

```
>>> random.random()
```

```
Traceback (most recent call last):
```

```
File "<pyshell#2>", line 1, in <module>
```

```
random.random()
```

```
NameError: name 'random' is not defined
```

- 如果**第一次**导入，则寻找模块源文件，**加载**模块，保存**模块对象**
- 导入的模块保存在字典**sys.modules**中
- **只导入一次**，再次import时仅仅添加相应的（模块等）对象引用。

模块的使用

- 可以使用dir函数查看任意模块中所有的对象列表，如果调用不带参数的dir()函数，则返回当前脚本的所有对象列表。
- 可以使用help函数查看任意模块或函数的使用帮助。

```
import sys
```

```
dir(sys)
```

```
help(sys)
```

```
>>> import math
>>> dir(math)
['_doc_', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh',
 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'comb', 'copysign', 'cos',
 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs', 'factoria
l', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose',
 'isfinite', 'isinf', 'isnan', 'isqrt', 'ldexp', 'lgamma', 'log', 'log10', 'log1
p', 'log2', 'modf', 'nan', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder',
 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc']
>>>
```

模块的使用

- **from 模块名 import 对象名[as 别名]**
 - 仅仅从模块中**导入特定的对象**，访问对象时**不再需要**包括模块名
 - 可以减少查询次数，提高执行速度

```
>>> from math import sin
```

```
>>> sin(3)
```

```
0.1411200080598672
```

```
>>> from math import sin as f #别名
```

```
>>> f(3)
```

```
0.1411200080598672
```

```
>>> from math import sin
```

```
>>> sin(3)
```

```
0.1411200080598672
```

```
>>> cos(3)
```

```
Traceback (most recent call last):
```

```
File "<pyshell#5>", line 1, in <module>
```

```
cos(3)
```

```
NameError: name 'cos' is not defined
```

```
>>> math.cos(3)
```

```
Traceback (most recent call last):
```

```
File "<pyshell#6>", line 1, in <module>
```

```
math.cos(3)
```

```
NameError: name 'math' is not defined
```

```
>>> from math import cos
```

```
>>> cos(3)
```

```
-0.9899924966004454
```

```
>>> |
```

模块的使用

- `from math import *`

- 从模块中导入所有的对象
- 多个模块中有同样的对象名时造成混乱，因此谨慎使用

- 内置模块**builtins**定义了解释器支持的内置函数，但是并不需要额外import

- 解释器内置执行了下述语句

```
import builtins as __builtins__
```

```
from builtins import *
```

```
>>> dir(__builtins__)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning', 'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError', 'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning', 'EOFError', 'Ellipsis', 'EnvironmentError', 'Exception', 'False', 'FileExistsError', 'FileNotFoundError', 'FloatingPointError', 'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError', 'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError', 'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError', 'MemoryError', 'NameError', 'None', 'NotADirectoryError', 'NotImplemented', 'NotImplementedError', 'OSError', 'OverflowError', 'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError', 'ReferenceError', 'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError', 'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError', 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning', 'ValueError', 'Warning', 'WindowsError', 'ZeroDivisionError', '_', '__build_class__', '__debug__', '__doctest__', '__import__', '__loader__', '__name__', '__package__', '__spec__', 'abs', 'all', 'any', 'ascii', 'bin', 'bool', 'bytearray', 'bytes', 'callable', 'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits', 'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'execfile', 'filter', 'float', 'format', 'frozenset', 'getattr', 'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'localize', 'map', 'max', 'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property', 'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars', 'zip']
>>>
```

到哪里去找到
math.py 文件？

自己写的程序.py

```
...  
...  
angle = 2.6  
...  
import math  
...  
y = math.sin(angle)  
...  
...  
...
```

运行 math.py

math.py

```
def sin(x):  
    ...  
    ...  
def cos(x):  
    ...  
    ...  
...  
...  
...
```

由于之前的 **import** 语句，
函数 **math.sin()** 在自己的程序中就有了定义，
现在就能使用该函数。

导入模块的另一种写法：
from math import sin
y = **sin**(angle)

导入时到哪里查找模块文件?(不作要求)

- 首先，在**当前文件夹**中查找模块文件；

```
>>> import os
```

```
>>> os.getcwd()#获得当前文件夹
```

```
>>> os.chdir( "d:\\Python程序" )#改变当前文件夹
```

- 导入模块时会从**sys.path**给出的目录列表中查找

```
>>> import sys
```

```
>>> sys.path
```

```
['', 'C:\\Program Files (x86)\\Python35\\python35.zip', 'C:\\Program Files (x86)\\Python35\\DLLs', 'C:\\Program Files (x86)\\Python35\\lib', 'C:\\Program Files (x86)\\Python35', 'C:\\Program Files (x86)\\Python35\\lib\\site-packages']
```

- 通过**append**自定义的目录到**sys.path**可以扩展搜索路径
- >>> sys.path.append('c:\\MyPythonLib')
- 注意搜索顺序可能导致没有导入正确的模块
- 在导入模块时，会优先导入相应的pyc文件，如果相应的pyc文件与py文件时间不相符，则导入py文件并重新编译该模块。

查看和改变path变量?(不作要求)

```
>>> import sys
>>> sys.path
['', 'C:\\Python34\\Lib\\idlelib', 'C:\\Windows\\system32\\python34.zip', 'C:\\Python34\\DLLs', 'C:\\Python34\\lib', 'C:\\Python34', 'C:\\Python34\\lib\\site-packages']
>>> sys.path.append('C:\\MyPythonLib')
>>> sys.path
['', 'C:\\Python34\\Lib\\idlelib', 'C:\\Windows\\system32\\python34.zip', 'C:\\Python34\\DLLs', 'C:\\Python34\\lib', 'C:\\Python34', 'C:\\Python34\\lib\\site-packages', 'C:\\MyPythonLib']
```

math模块

- **import math** 或者 from math import *
- math.pi: 数学常量 π print(math.pi) = 3.141592653589793
- math.e: 数学常量e math.e = 2.718281828459045

函数	含义	实例	结果
fabs(x)	绝对值, 返回float	fabs(-3)	3.0
ceil(x)	大于等于x的最小的整数	ceil(1.2),ceil(-1.6)	(2,-1)
floor(x)	小于等于x的最大的整数	floor(1.8),floor(-2.1)	(1,-3)
trunc(x)	截取为 最接近0 的整数	trunc(1.2),trunc(-2.8)	(1,-2)
factorial(x)	整数x(≥ 0)的阶乘	factorial(5)	120
sqrt(x)	x(≥ 0)的平方根	sqrt(2)	1.4142135623730951
exp(x)	$e^{**}x$	exp(5)	148.4131591025766
log(x[,base])	以base为底的对数, base没有则为以e为底的自然对数	log(math.e**2), log(4,2)	(2.0,2.0)

math模块:三角函数和角度转换

函数	含义	实例	结果
<code>sin(x)</code>	x的正弦	<code>sin(pi/2)</code>	1.0
<code>cos(x)</code>	x的余弦	<code>cos(2*pi)</code>	1.0
<code>tan(x)</code>	x的正切	<code>tan(pi/4)</code>	0.9999999999999999
<code>asin(x)</code>	x的反正弦	<code>asin(1)</code>	1.5707963267948966
<code>acos(x)</code>	x的反余弦	<code>acos(1)</code>	0.0
<code>atan(x)</code>	x的反正切	<code>atan(1)</code>	0.7853981633974483
<code>degrees(x)</code>	x从弧度转换为角度	<code>degrees(pi)</code>	180.0
<code>radians(x)</code>	x从角度转换为弧度	<code>radians(90)</code>	1.5707963267948966

random模块

```
import random
```

```
r=random.randint(0,25) # 生成的随机数 n: 0 <= n <= 25
```

```
print("The character is",chr(r+ord('a'))) # 随机小写英文字母
```

```
import string
```

```
print("The character is",random.choice(string.ascii_lowercase))
```

```
print(random.sample(range(1,100),4)) #1到100之间随机4个不同的整数
```

```
random.randrange(10, 100, 2), 结果相当于从[10, 12, 14, 16, ... 96, 98]序列中获取一个随机数
```

函数	含义
random()	返回在[0,1) 区间的随机实数
uniform(a,b)	返回在[a,b]区间的随机实数
randint(a,b)	返回在[a,b]区间的随机整数,包括a,b
choice(seq)	返回序列seq中的随机一个元素
sample(seq,n)	从 序列seq中随机选择 n个不同的元素
random.randrange([start], stop[, step]),	从指定范围内, 按指定基数递增的集合中 获取一个随机数,不包括stop, start默认为0

模块的使用

- **import 模块名 [as 别名]**

```
>>>import math
```

```
>>>math.sin(0.5)          #求0.5的正弦
```

```
>>>import random as rnd
```

```
>>>x=rnd.random( )  #获得[0,1) 内的随机小数
```

- **from 模块名 import 对象名[as 别名]**

```
>>> from math import sin
```

```
>>> sin(3)
```

```
>>> from math import sin as f #别名
```

```
>>> f(3)
```

- **From 模块名 import ***

```
>>> from math import *
```

```
>>> sin(3)
```

```
>>> cos(3)
```

练习一

```
>>> import math
```

```
>>> math.sin(0.5)
```

```
0.479425538604203
```

```
>>> import random
```

```
>>> n = random.random()
```

```
>>> n = random.randint(1,100)
```

```
>>> n = random.randrange(1, 100)
```

#导入标准库math

#求0.5（单位是弧度）的正弦

#导入标准库random

#获得[0,1) 内的随机小数

#获得[1,100]区间上的随机整数

#返回[1, 100)区间中的随机整数

练习二

```
>>> from math import sin
```

#只导入模块中的指定对象，访问速度略快

```
>>> sin(3)
```

```
0.1411200080598672
```

```
>>> from math import sin as f
```

#给导入的对象起个别名

```
>>> f(3)
```

```
0.1411200080598672
```

练习三

```
>>> from math import *
>>> sin(3)
0.1411200080598672
>>> gcd(36, 18)
18
>>> pi
3.141592653589793
>>> e
2.718281828459045
>>> log2(8)
3.0
>>> log10(100)
2.0
>>> radians(180)
3.141592653589793
```

#导入标准库math中所有对象
#求正弦值

#最大公约数

#常数 π

#常数e

#计算以2为底的对数值

#计算以10为底的对数值

#把角度转换为弧度

MyModule.py

编写模块

```
def fahrenheit_from_Celsius(Celsius):  
  
    """摄氏度转换到华氏度"""  
  
    Fahrenheit = (9 / 5) * Celsius + 32  
    return Fahrenheit
```

```
#主程序  
temperature_cel = 35  
temperature_fah = fahrenheit_from_Celsius(temperature_cel)  
print(temperature_fah)
```

这里的主程序
也可以看成自己测试
代码

```
>>>  
===== RESTART: /Users/xiaochunxiao/Desktop/myModule.py =====  
95.0
```

果然正确！

callmyModule.py

因为模块文件名为 MyModule.py

别人可能希望使用你编写的函数

```
import myModule #我想用一下你编好的函数
```

```
inputCel = float(input("Please input Celsius:"))  
resultFah = myModule.fahrenheit_from_Celsius(inputCel)  
print(inputCel,"摄氏度 为 ",resultFah,"华氏度")
```

```
===== RESTART: /Users/xiaochunxiao/Desktop/callmyModule.py =====  
95.0  
Please input Celsius:45  
45.0 摄氏度 为 113.0 华氏度  
>>> |
```

这里myModule运行的内容就显得很突兀了

1.7 __name__属性

- 每个Python程序在运行时都有一个 “__name__” 属性。
 - 如果程序作为模块被导入，则其 “__name__” 属性的值被自动设置为模块名
 - 如果程序独立运行，称为脚本(script)，则其 “__name__” 属性值被自动设置为 “__main__” 。

例如，假设文件nametest.py中只包含下面一行代码：

```
print(__name__)
```

- 直接运行时：

__main__

- 而将该文件作为模块导入时得到如下执行结果：

```
>>> import nametest
```

```
nametest
```

```
if __name__ == '__main__':  
    main()
```

例子：

myModule.py

```
def fahrenheit_from_Celsius(Celsius):
```

```
    """摄氏度转换到华氏度"""
```

```
    Fahrenheit = (9 / 5) * Celsius + 32
```

```
    return Fahrenheit
```

```
#我来测试一下我写的函数对吗
```

```
temperature_cel = 35
```

```
temperature_fah = fahrenheit_from_Celsius(temperature_cel)
```

```
print(temperature_fah)
```

```
print("myModule的名字是: ", __name__)
```

```
===== RESTART: /Users/xiaochunxiao/Desktop/myModule.py =====
```

```
95.0
```

```
myModule的名字是: __main__
```

```
===== RESTART: /Users/xiaochunxiao/Desktop/callmyModule.py =====
```

```
95.0
```

```
myModule的名字是: myModule
```

```
Please input Celsius:
```

callmyModule.py

```
import myModule #我想用一下你编好的函数
```

```
inputCel = float(input("Please input Celsius:"))
```

```
resultFah = myModule.fahrenheit_from_Celsius(inputCel)
```

```
print(inputCel, "摄氏度 为 ", resultFah, "华氏度")
```

修改模块 myModule.py

myModule.py

```
def fahrenheit_from_Celsius(Celsius):  
  
    """摄氏温度转换到华氏度"""  
    Fahrenheit = (9 / 5) * Celsius + 32  
    return Fahrenheit
```

#我来测试一下我写的函数对吗

```
def main():  
    temperature_cel = 35  
    temperature_fah = fahrenheit_from_Celsius(temperature_cel)  
    print(temperature_fah)  
    print("myModule的名字是: ", __name__)
```

```
if __name__ == "__main__":#有条件执行, 就是作为主程序时才运行  
    main()#函数只有调用才能执行
```

```
===== RESTART: /Users/xiaochunxiao/Desktop/myModule.py =====
```

95.0

myModule的名字是: __main__

1.将所有测试代码放在一个函数中, 通常命名为main()。

2.将main()放在条件语句中, 既不影响执行自测功能, 也不影响该模块被调用。

再次运行callmyModule模块：

```
import myModule #我想用一下你编好的函数
```

```
inputCel = float(input("Please input Celsius:"))  
resultFah = myModule.fahrenheit_from_Celsius(inputCel)  
print(inputCel,"摄氏度 为 ",resultFah,"华氏度")|
```

```
===== RESTART: /Users/xiaochunxiao/Desktop/callmyModule.py =====
```

```
Please input Celsius:45
```

```
45.0 摄氏度 为  113.0 华氏度
```

```
>>>
```

使用main函数的程序结构

```
import math
```

```
.....
```

```
def fun1(.....):  
    .....|
```

```
def fun2(.....):  
    .....
```

```
.....
```

```
def main():  
    .....
```

```
if __name__ == "__main__":  
    main()
```

*我们以后的作业都希望使用这种
具有main函数的格式！！！！*

1.8 编写自己的包 (package)(不作要求)

- 按照目录来组织模块，称为包(package)
 - 避免模块名字相同
 - 维护模块之间的联系
- 包的每个目录（比如foo)中都必须包含一个__init__.py文件
 - 用于表示该目录是一个包。如果没有，被当作一个普通的目录
 - 该文件甚至可以是一个空文件
 - __init__.py也是一个模块，模块名为foo
 - 一般__init__.py文件的主要用途是设置__all__变量以及所包含的包初始化所需的代码。其中__all__变量中定义的对象可以在使用from ... import *时全部正确导入。
 - 可以有多级目录，组成多级层次的包结构

1.8 编写自己的包 (package))(不作要求)

foo/		模块
__init__.py	-----	foo
abc.py	-----	foo.abc
rst.py	-----	foo.rst
bar1/		
__init__.py	-----	foo.bar1
abc.py	-----	foo.bar1.abc
uvw.py	-----	foo.bar1.uvw
bar2/		
__init__.py	-----	foo.bar2
abc.py	-----	foo.bar2.abc
xyz.py	-----	foo.bar2.xyz

```
import foo.bar2.xyz
foo.bar2.xyz.func1(...)
```

1.6 Python文件名

- `.py` : Python源文件, 由Python解释器负责解释执行。
- `.pyw` : Python源文件, 常用于图形界面程序文件。
- `.pyc` : Python字节码文件, 可用于隐藏Python源代码和提高运行速度。对于Python模块, 第一次被导入时将被编译成字节码的形式, 并在以后再次导入时优先使用 “.pyc” 文件
- `.pyo` : 优化的Python字节码文件, 使用 “python -O -m py_compile file.py” 或 “python -OO -m py_compile file.py” 进行优化编译。
- `.pyd` : 一般是由其他语言编写并编译的二进制文件, 常用于实现某些软件工具的Python编程接口插件或Python动态链接库。